# SC4001 Neural Networks and Deep Learning Assignment: Part (a)

**Aaron Soh**
College of Computing and Data Science
Nanyang Technological University
Singapore
asoh014@e.ntu.edu.sg

## Abstract

In part (a) of the assignment, we are tasked to develop deep neural networks (DNNs) to predict Singapore HDB resale prices. We train a simple deep neural network with varying hyperparameters and evaluate its performance in this report.

## 1 Generative AI Declaration

In this assignment, I used ChatGPT to assist with the debugging of environment issues related to experiment rollout using the Ray package. All AI-generated bug fixes was carefully reviewed, modified where necessary, integrated into the overall solution by me, and tested to ensure it met the assignment requirements. No other AI was used in the other parts of the assignment. The final submitted work represents my own understanding and effort. I take full responsibility for the accuracy, functionality, and academic integrity of the submitted solution.

**Generative AI Tool Used**: ChatGPT

**Purpose of Use**: Bug fix for Part A1 involving hyperparameter search with Ray

**Verification Method**: Manual code review

## 2 Exploratory Data Analysis

In this section, we analyse the given dataset `hdb_price_prediction.csv`. Through simple EDA and analysis of the provided baseline code, we identified categorical and continuous variables.

Of the 4 categorical variables, there are a total of 26 towns, 44 flat types, and 17 different storey ranges. The remaining 6 variables (specified in `config.py`) are continuous, and our goal is to predict `resale_price`. As instructed, we use years 2017-2020 for training, 2021 for validation and model selection, and 2022 for testing.

Table 1 shows the statistics of the provided dataset. The dataset contains 16,624 rows of 2023 data, which we exclude in this experiment.

Table 1: Dataset split

| Split | Size |
|---|---|
| Train | 87,370 |
| Validation | 29,057 |
| Test | 26,702 |
| Extra | 16,424 |

Preprint.

### 2.1 Preprocessing

To convert our categorical features into trainable data, we first initialise our embedding table. We use a fixed dimension 8 for each embedding layer, and the number of embeddings generated is equal to the cardinality of each categorial variable, plus 1 for the hidden class.

## 3 Question A1

### 3.1 Architecture

We first calculate the dimensions of our embedding tables. The input width is calculated as the product of the embedding dimension with the number of categorical variables, summed with the number of continuous inputs. We obtain an input width of 38 in this scenario.

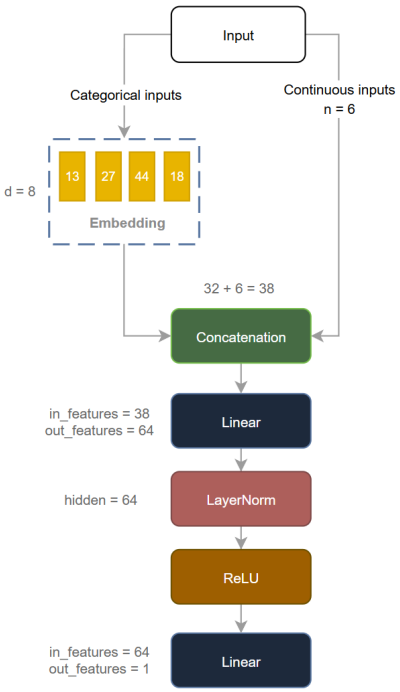



Figure 1: Block diagram of the baseline architecture. Total number of trainable parameters = 3505.

Figure 1 shows the block diagram of our baseline architecture. Some simple math will give us the following calculation for the total number of parameters for the baseline network:

$$(13 + 27 + 44 + 18) \times 8 = 816 \text{ embedding parameters}$$

For the first linear layer,

$$38 \times 64 \text{ weights} + 64 \text{ bias} = 2496$$

LayerNorm:

$$64 + 64 = 128$$

Final output layer:

$$64 + 1 = 65$$

Hence, the total number of parameters is $816 + 2496 + 128 + 65 = 3505$.

### 3.2 Finding the optimal hyperparameters

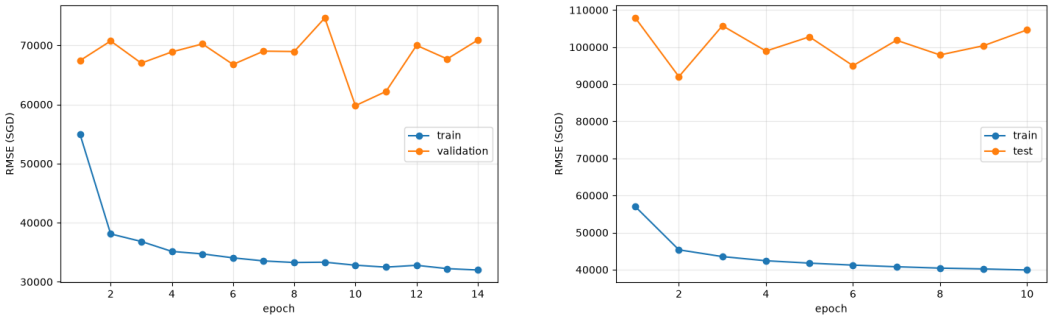
We use the Ray library to find the optimal hidden layer width and embedding dimension. The given search space is {16, 32, 64, 128, 256} and {2, 4, 8, 12, 16} respectively. This gives us 25 possible configurations to test. We evaluate the RMSE on the validation set.

Table 2: Hyperparameter Search Results (RMSE)

| **hidden_width \ embedding_dim** | **2** | **4** | **8** | **12** | **16** |
|---|---|---|---|---|---|
| 16 | 66426 | 68880 | 67520 | 63026 | 67377 |
| 32 | 69204 | 62763 | 66491 | 62953 | 64193 |
| 64 | 63365 | 64524 | 63412 | 64197 | 63417 |
| 128 | 66356 | 61533 | 67248 | 60037 | 65642 |
| 256 | **59792** | 61084 | 63579 | 65522 | 63147 |

Table 2 shows the RMSE results from the hyperparameter search using the Ray library. The best performing model uses hidden_width=256 and embedding_dim=2, achieving a validation RMSE of 59792.

On retraining with the combined train and validation set, evaluating on the test set results in a final RMSE of 104,637 and $R^2$ value of 0.6223.




(a) Train and validation loss against epoch.

(b) Train+val and test loss against epoch.

Figure 2: Loss curves for the optimal hyperparameters hidden_width=256 and embedding_dim=2.

Looking at the loss curves, we notice that the test and validation RMSE curves are very noisy while train RMSE falls smoothly. This is a sign that our model is overfitting to the training data, and unable to generalise to unseen data since there is essentially no improvement with each passing epoch.

The observations are expected as we are using old data to predict prices in later years, which may not be very correlated. As the saying goes, past performance is not indicative of future results.

## 4 Question A2

In this section, we perform ablation experiments on our baseline model and the model with hidden_width=256 and embedding_dim=2. Note that this configuration bumps our number of trainable parameters up to 4,813. For this section, we use the seeds {67, 420, 69, 21, 13} for our 5 experiments. We use the same preprocessing steps and hyperparameters as part **A1**.

### 4.1 Removing the categorical features

Taking only the continuous features as input, this reduces our input width size from 38 to 6. Running across all 5 seeds, we compute the mean and standard deviation on the test RMSE and compare it against the baseline. Table 3 shows the number of parameters, test RMSE and test $R^2$ of the ablation model against the baseline.

Table 3: Model performance with and without categorical features

| model | params | test RMSE $\pm$ std dev | test $R^2 \pm$ std dev |
|---|---|---|---|
| baseline | 4,813 | **97,965 $\pm$ 4,321** | **0.6684 $\pm$ 0.0294** |
| ablation | 2,765 | 111,002 $\pm$ 5,502 | 0.5741 $\pm$ 0.0423 |

We observe that removing the embedding table for the categorical variables results in much worse performance. This gives strong evidence that the 4 categorical variables are strong features that help our model to better predict housing prices, and should not be removed (even though they take up near half the parameter count).

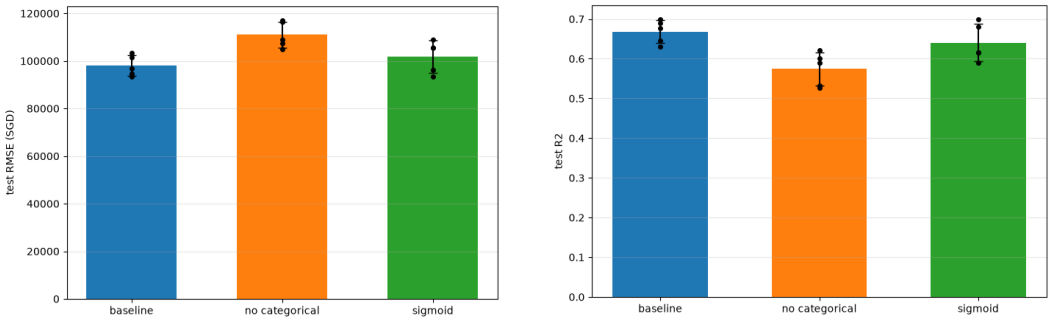
### 4.2 Replacing ReLU with Sigmoid

Table 4: Model performance with ReLU and Sigmoid

| activation | params | test RMSE $\pm$ std dev | test $R^2 \pm$ std dev |
|---|---|---|---|
| Baseline (ReLU) | 4,813 | **97,965 $\pm$ 4,321** | **0.6684 $\pm$ 0.0294** |
| Sigmoid | 4,813 | 101,921 $\pm$ 6,787 | 0.6404 $\pm$ 0.0473 |

From Table 4, we observe that swapping ReLU to sigmoid causes the model to perform worse as well. One possible explanation is that it avoids the vanishing gradient problem, something that the sigmoid activation struggles with, causing training to be more unstable and results in a worser model.

### 4.3 Findings




(a) Box plot of test RMSE.

(b) Box plot of test $R^2$.

Figure 3: Box plot of test RMSE and $R^2$ for the baseline and both ablation models.

From Figure 3, we observe that both ablations perform worse than the baseline model. Clearly, there is value in retaining the embedding table, but the difference in RMSE and $R^2$ for the activation function ablation is still within the standard deviation of its own measurement - hence it is not a very convincing signal that one activation function is better than the other.

## 5 Question A3

Next, we evaluate the **wide-and-deep** architecture proposed in the assignment and sweep over different values of $\lambda = \{0, 0.25, 0.5, 1, 2\}$. Figure 4 shows the model architecture defined in the assignment. In this wide-and-deep architecture, the output is obtained by a linear sum of the outputs of the two branches:

$$\text{output} = \text{MLP output} + \lambda \times \text{linear output}$$

Figure 5 shows the results of the experiment. All experiments were run with the same set of 5 seeds used in **A2**, for a maximum of 20 epochs with early stopping and patience 4 with minimum

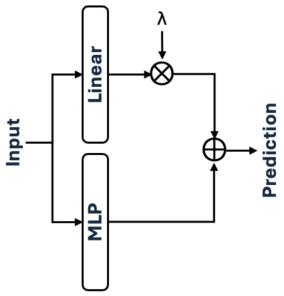
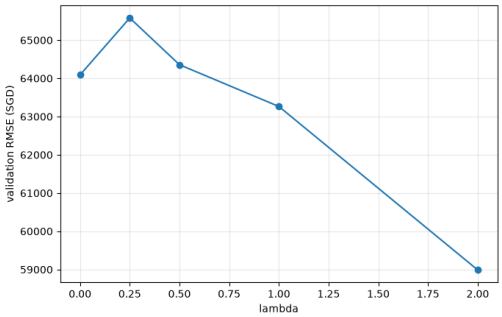



Figure 4: Wide and deep architecture

improvement 1e-4. The results favour a larger value of $\lambda$, as it achieves the lowest validation RMSE. This suggests that increasing the weightage of the linear branch seems to improve performance, as it outperforms the baseline validation RMSE of 59,792 found in Section 2.2. Nonetheless, from Table 5, the baseline still performs better on the test set.




| lambda | best epoch | validation RMSE |
|---|---|---|
| 0.00 | 11 | 64090 |
| 0.25 | 7 | 65580 |
| 0.50 | 12 | 64357 |
| 1.00 | 6 | 63264 |
| 2.00 | 10 | **58986** |

Figure 5: Validation RMSE across different values of $\lambda$.

Table 5: Wide and deep architecture on the test set.

| model | params | test RMSE $\pm$ std dev | test $R^2 \pm$ std dev |
|---|---|---|---|
| A1 Baseline | 4,813 | **97,965 $\pm$ 4,321** | **0.6684 $\pm$ 0.0294** |
| wide and deep | 4,813 | 98,148 $\pm$ 5,521 | 0.6669 $\pm$ 0.0385 |

In theory, the wide and deep architecture tries to exploit a shorter gradient path to balance memorisation and generalisation in machine learning tasks (Cheng et al., 2016). The *wide* portion, which is the linear branch, is cheaper to train in principle since it can model certain relationships that are assumed to be linear via a linear function.

Also, we note that the wide-and-deep architecture was originally designed for *sparse recommender systems*, but our task here involves *dense* regression to predict future housing prices. The wide branch can only capture simple linear dependencies which may collapse due to a time shift (or distribution shift, in our case) **since the years are different**. Because *year* isn't an input feature, one theory that we have is that the wide branch cannot learn to modulate its memorised rules based on the time period and ends up adding **noise** to the predictions.

Note that one limitation is that the RMSE standard deviation is quite large, so the differences across the $\lambda$ sweep are not statistically distinguishable. Our chosen $\lambda$ may therefore reflect run-to-run variability rather than a true optimum.

# 6 Question A4

We retrain both the **A1 baseline** model and the **A3 wide-and-deep** model on the same default seed 42. For each model, we computed the FeatureAblation attribution using the **Captum** library to determine the global feature importance score for all of our 10 input features. Figure 6 shows a labelled bar plot of the feature importances, sorted in descending order based on their mean absolute attribution in Singapore Dollars (SGD).

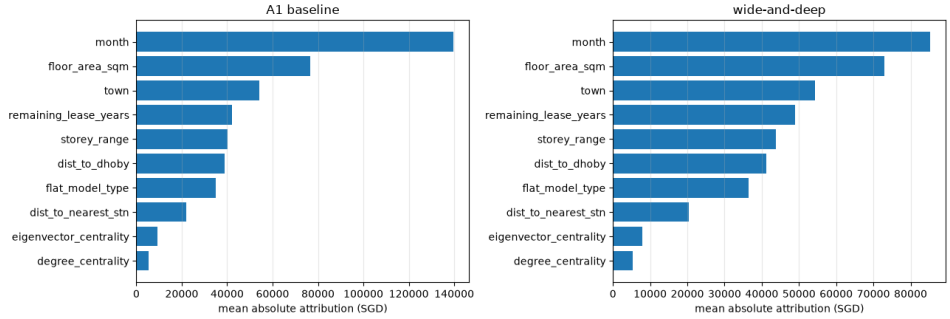



Figure 6: Feature importance in descending order by mean absolute attribution in Singapore dollars.

We observe that both models have exactly identical ordering of feature importances, with minor differences in the mean absolute attribution. For the A1 baseline, the top three are **month** (139,541 SGD), **floor_area_sqm** (76,420) and **town** (54,267); for the wide-and-deep they are the same: **month** (85,157), **floor_area_sqm** (73,043) and **town** (54,320). Intuitively, these make sense since bigger houses and more central towns naturally cost more.

However, the interesting observation is the *month*. Despite being the top feature for both models, the magnitude drops by 54,384 SGD in the wide-and-deep architecture. This difference likely suggests how the wide branch distributes the credit to each feature more evenly. The direct linear path from the month embedding to the output appropriately models the sesasonal effect of housing prices.

## 6.1 Limitations

FeatureAblation measures a feature's importance by removing that feature and seeing how much the prediction changes. For categorical features, the FeatureAblation package replaces a column with 0. However, zero is an index into an embedding table which corresponds to an "unseen" category that never appears in training. Because no training example uses that category, that embedding row never receives a gradient update and stays at its random initialisation.

This highlights an issue with the study because we are just swapping in a random vector, resulting in an arbitrary difference when we try to compute that feature's importance. Meanwhile, for continuous variables, it is quite clear that the baseline should be a value of 0. Perhaps, for a better ablation study, we could have retrained the models but remove a categorical variable one at a time to study the impact of its absence.

# 7 Credits

This report was written in LaTeX using the NeurIPS 2026 style files from `https://neurips.cc`. We also would like to credit `https://github.com/WangX0111/drawio-nn-templates` for the draw.io templates used to draw the block diagram for this assignment.

# References

Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide & Deep Learning for Recommender Systems. *arXiv preprint arXiv:1606.07792*, 2016.